Sécurisation d'une application de gestion des utilisateurs avec hachage de mot de passe, gestion des sessions, redirection selon les rôles et protection contre les injections SQL

Partie 1- base de données gestion des utilisateurs, formulaire d'inscription, connexion

Objectifs

- Développer une base de données sécurisée pour la gestion des utilisateurs avec des rôles (admin, user).
- Implémenter un formulaire d'inscription sécurisé avec hachage des mots de passe pour garantir leur sécurité.
- Utiliser les sessions PHP pour maintenir l'état de l'utilisateur connecté et rediriger vers des pages protégées selon le rôle.
- Sécuriser l'application contre les attaques par injection SQL grâce à l'utilisation de requêtes préparées et de liens sécurisés.

Contexte

Vous allez développer une petite application web qui permet aux utilisateurs de s'inscrire et de se connecter pour accéder à des pages protégées. L'application est sécurisée contre les attaques par **injection SQL** et les attaques de **bypass d'authentification**. L'utilisateur doit être redirigé vers des pages spécifiques selon son rôle (*administrateur* ou *utilisateur classique*) après une connexion réussie.

Lors de l'inscription, le mot de passe de l'utilisateur est **haché** pour être stocké de manière sécurisée dans la base de données.

1. Création de la base de données et de la table utilisateurs

Voici le script SQL pour créer la table des utilisateurs :

```sql
-- Créer la table utilisateurs
CREATE TABLE utilisateurs (
    id INT AUTO_INCREMENT PRIMARY KEY,   -- Identifiant unique
    nom VARCHAR(100) NOT NULL,            -- Nom de l'utilisateur
    email VARCHAR(100) NOT NULL UNIQUE,   -- Email unique de l'utilisateur
    mot_de_passe VARCHAR(255) NOT NULL,   -- Mot de passe crypté
    role VARCHAR(20) NOT NULL,            -- Rôle de l'utilisateur (admin, user)
    date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- Date de création du compte
);
```

2. Formulaire d'inscription sécurisé avec hachage du mot de passe

Lorsqu'un utilisateur s'inscrit, son mot de passe doit être haché avant d'être stocké dans la base de données pour garantir la sécurité. Voici un formulaire d'inscription **(inscription.php)** en PHP avec la gestion sécurisée du mot de passe.

```php
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Connexion à la base de données
    $pdo = new PDO("mysql:host=localhost;dbname=gestion_utilisateurs", "root", "");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Récupérer les données du formulaire
    $nom = $_POST['nom'];
    $email = $_POST['email'];
    $motdepasse = $_POST['motdepasse'];
    $role = $_POST['role'];  // Rôle, 'admin' ou 'user'

    // Hacher le mot de passe avant de l'insérer
    $motdepasse_hache = password_hash($motdepasse, PASSWORD_DEFAULT);

    // Préparer la requête SQL pour l'insertion
    $stmt = $pdo->prepare("INSERT INTO utilisateurs (nom, email, mot_de_passe, role) VALUES (:nom, :email, :motdepasse, :role)");
    $stmt->bindParam(':nom', $nom);
    $stmt->bindParam(':email', $email);
    $stmt->bindParam(':motdepasse', $motdepasse_hache);
    $stmt->bindParam(':role', $role);
```

```php
        // Exécuter la requête
        if ($stmt->execute()) {
            echo "Inscription réussie ! Vous pouvez maintenant vous connecter.";
        } else {
            echo "Erreur lors de l'inscription. Veuillez réessayer.";
        }
    }
    ?>

<form method="POST">
    Nom : <input type="text" name="nom" required><br>
    Email : <input type="email" name="email" required><br>
    Mot de passe : <input type="password" name="motdepasse" required><br>
    Rôle :
    <select name="role">
        <option value="admin">Admin</option>
        <option value="user">Utilisateur</option>
    </select><br>
    <input type="submit" value="S'inscrire">
</form>
```

3. Authentification sécurisée avec hachage et gestion des sessions

Lors de la connexion de l'utilisateur, son mot de passe est comparé à celui stocké dans la base de données grâce à la fonction **password_verify().**

Connexion.php

```php
<?php
session_start(); // Démarrer la session

// Connexion sécurisée à la base de données
$pdo = new PDO("mysql:host=localhost;dbname=gestion_utilisateurs", "root", "");
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

// Vérification des données du formulaire de connexion
if (isset($_POST['email']) && isset($_POST['motdepasse'])) {
    $email = $_POST['email'];
    $motdepasse = $_POST['motdepasse'];

    // Requête préparée pour sécuriser l'authentification
    $stmt = $pdo->prepare("SELECT * FROM utilisateurs WHERE email = :email");
    $stmt->bindParam(':email', $email, PDO::PARAM_STR);
    $stmt->execute();

    $user = $stmt->fetch(PDO::FETCH_ASSOC);
```

```php
    // Vérifier si l'utilisateur existe et le mot de passe est valide
    if ($user && password_verify($motdepasse, $user['mot_de_passe'])) {
        // Stocker les informations de l'utilisateur dans la session
        $_SESSION['id'] = $user['id'];
        $_SESSION['nom'] = $user['nom'];
        $_SESSION['role'] = $user['role'];

        // Rediriger en fonction du rôle de l'utilisateur
        if ($user['role'] == 'admin') {
            header('Location: page_secrete1.php'); // Redirection vers la page admin
            exit();
        } else {
            header('Location: page_secrete2.php'); // Redirection vers la page utilisateur
            exit();
        }
    } else {
        // Si l'utilisateur n'existe pas ou les informations sont incorrectes
        echo "Identifiant ou mot de passe incorrect.";
    }
} else {
    echo "Veuillez entrer vos identifiants.";
}
?>
```

```php
?>

<form method="POST">
    Email : <input type="email" name="email" required><br>
    Mot de passe : <input type="password" name="motdepasse" required><br>
    <input type="submit" value="Se connecter">
</form>
```